

Problem Statement

Company ABC has recently hired a new employee named Alex Bob. As the Salesforce developer, you are tasked with printing the details for Alex.

Requirements

- Employee Name
- Employee Salary
- Employee Number
- Employee Joining Date
- Is Employee being Experienced?

```
public class Employee {  
  
    public String employeeName;  
  
    public Double employeeSalary;  
  
    public String employeeNumber;  
  
    public Date employeeJoiningDate;  
  
    public Boolean isExperienced;  
  
    public Employee(String name, Decimal salary, String empNumber, Date joinDate, Boolean  
experienced) {  
  
        this.employeeName = name;  
  
        this.employeeSalary = salary;
```

```
this.employeeNumber = empNumber;

this.employeeJoiningDate = joinDate;

this.isExperienced = experienced;

}

public void printEmployeeDetails() {

    System.debug('Employee Name: ' + employeeName);

    System.debug('Employee Salary: ' + employeeSalary);

    System.debug('Employee Number: ' + employeeNumber);

    System.debug('Employee Joining Date: ' + employeeJoiningDate);

    System.debug('Is Employee Experienced: ' + isExperienced);

}

}

Employee alexBob = new Employee('Alex Bob', 75000.00, 'EMP12345', Date.newInstance(2025, 1,
1), true);
alexBob.printEmployeeDetails();
```

Execution Log		
Timestamp	Event	Details
09:55:56:016	USER_DEBUG	[29] DEBUG Employee Name: Alex Bob
09:55:56:016	USER_DEBUG	[31] DEBUG Employee Salary: 75000.0
09:55:56:016	USER_DEBUG	[33] DEBUG Employee Number: EMP12345
09:55:56:016	USER_DEBUG	[35] DEBUG Employee Joining Date: 2025-01-01 00:00:00
09:55:56:016	USER_DEBUG	[37] DEBUG Is Employee Experienced: true

Problem Statement

You are tasked with designing an inventory management system for a small retail store. The store sells various products, including electronics, clothing, and accessories. The current system lacks organization, and the store owner wants to improve efficiency.

Requirements

You need to complete the below job as part of the given task:

- Add new products to the catalogue (List).
 - Add categories (ex: Electronics, Clothing) (Set).
 - Adding new products with initial quantities (Map).
-

```
public class Inventory {  
  
    public void catalogue() {  
  
        list<string> products = new list<string>();  
  
        products.add('Headphones');  
        products.add('Speakers');  
        products.add('Jacket');  
        products.add('Dress');
```

```
        system.debug(products);  
    }
```

```
public void categories() {
```

```
    set<string> st = new set<string>();  
    st.add('Electronics');  
    st.add('Clothing');  
    st.add('Accessories');  
    st.add('Home');
```

```
        system.debug(st);  
    }
```

```
public void quantities() {
```

```
    map<string, integer> mp = new map<string, integer>();  
    mp.put('Electronics', 40);  
    mp.put('Clothing', 100);  
    mp.put('Accessories', 60);  
    mp.put('Home', 40);
```

```
        system.debug(mp);  
    }  
}
```

```
Inventory i = new Inventory();  
i.catalogue();
```

```
i.categories();  
i.quantities();
```

Execution Log		
Timestamp	Event	Details
10:01:31:022	USER_DEBUG	[12]]DEBUG[(Headphones, Speakers, Jacket, Dress)
10:01:31:022	USER_DEBUG	[23]]DEBUG[{Accessories, Clothing, Electronics, Home}
10:01:31:023	USER_DEBUG	[35]]DEBUG[{Accessories=60, Clothing=100, Electronics=40, Home=40}

Problem Statement

Create a CustomerOrder class that encapsulates order-related information, such as items, number of items and total amount.

Implement methods to add items, number of items and to calculate the total.

Properties:

- items: A private string representing the name of the ordered item.
- totalItems: A private integer representing the quantity of the ordered item.
- totalAmount: A private decimal value representing the total amount for the order.
-

Methods:

- setItem(String itemName, Integer quantity, Decimal pricePerUnit).
- getCalculateTotal(): A public method that retrieves the total amount for the order. It returns the calculated total amount (decimal).

```
public class CustomerOrder {  
  
    private String items;  
    private Integer totalItems;  
    private Decimal totalAmount;  
  
    public CustomerOrder() {
```

```
    this.totalItems = 0;
    this.totalAmount = 0.00;
    this.items = "";
}
```

```
public void setItem(String itemName, Integer quantity, Decimal pricePerUnit) {
    this.items = itemName;
    this.totalItems = quantity;
    this.totalAmount = quantity * pricePerUnit;
}
```

```
public Decimal getCalculateTotal() {
    return this.totalAmount;
}
```

```
public Integer getTotalItems() {
    return this.totalItems;
}
```

```
public String getItemName() {
    return this.items;
}
}
```

```

CustomerOrder order = new CustomerOrder();
order.setItem('Laptop', 2, 1000.00);
Decimal total = order.getCalculateTotal();

System.debug('Item Ordered: ' + order.getItemName());
System.debug('Total Quantity: ' + order.getTotalItems());
System.debug('Total Amount: ' + total);

```

Execution Log		
Timestamp	Event	Details
10:06:48:019	USER_DEBUG	[5] DEBUG Item Ordered: Laptop
10:06:48:020	USER_DEBUG	[6] DEBUG Total Quantity: 2
10:06:48:020	USER_DEBUG	[7] DEBUG Total Amount: 2000.00

Problem Statement

Design a simple banking application that models bank accounts and savings accounts. The system should allow users to create bank accounts, deposit money, and calculate interest for savings accounts.

Requirements:

BankAccount (Base Class): Set Properties as accountNumber, balance, ownerName, amount.

- Implement a method called accountDetails(Integer accNumber, Decimal initialBalance, String owner, Decimal depositAmount).
- Define a method called totalAmount().

SavingsAccount (Subclass):

- Implement a method called calculateInterest(Decimal interest): This method calculates the interest earned based on the total amount and the provided interest rate.

```
public virtual class BankAccount {

    private Integer accountNumber;

    private Decimal balance;

    private String ownerName;

    private Decimal amountDeposited;


    public BankAccount(Integer accNumber, Decimal initialBalance, String owner, Decimal
depositAmount) {

        this.accountNumber = accNumber;

        this.balance = initialBalance;

        this.ownerName = owner;

        this.amountDeposited = depositAmount;

    }


    public void accountDetails(Integer accNumber, Decimal initialBalance, String owner, Decimal
depositAmount) {

        this.accountNumber = accNumber;
```

```
this.balance = initialBalance;
```

```
this.ownerName = owner;
```

```
this.amountDeposited = depositAmount;
```

```
this.balance += depositAmount;
```

```
System.debug('Account Number: ' + accountNumber);
```

```
System.debug('Owner: ' + ownerName);
```

```
System.debug('Initial Balance: ' + initialBalance);
```

```
System.debug('Amount Deposited: ' + depositAmount);
```

```
System.debug('Total Balance: ' + balance);
```

```
}
```

```
public Decimal totalAmount() {
```

```
    return this.balance;
```

```
}
```

```
public void displayAccountDetails() {
```

```
System.debug('Account Number: ' + accountNumber);
```

```
System.debug('Owner: ' + ownerName);
```

```
System.debug('Balance: ' + balance);
```

```
}
```

```
}
```

```
public class SavingsAccount extends BankAccount {
```

```
    public SavingsAccount(Integer accNumber, Decimal initialBalance, String owner, Decimal  
    depositAmount) {
```

```
        super(accNumber, initialBalance, owner, depositAmount); // Calls the parent class constructor
```

```
}
```

```
    public Decimal calculateInterest(Decimal interestRate) {
```

```
        Decimal interest = this.totalAmount() * (interestRate / 100);
```

```
        Decimal newBalance = this.totalAmount() + interest;
```

```
System.debug('Interest earned: ' + interest);
```

```
System.debug('New Balance after interest: ' + newBalance);
```

```
return interest;
```

```
}
```

```
}
```

```
37 SavingsAccount savingsAccount = new SavingsAccount(1001, 5000.00, 'John Doe', 1500.00);
38 savingsAccount.accountDetails(1001, 5000.00, 'John Doe', 1500.00);
39 Decimal interestEarned = savingsAccount.calculateInterest(5.0);
40 Decimal totalBalance = savingsAccount.totalAmount();
41 System.debug('Total Balance after Interest: ' + totalBalance);
42 savingsAccount.displayAccountDetails();
```

Execution Log		
Timestamp	Event	Details
10:18:48:048	USER_DEBUG	[43] DEBUG Account Number: 1001
10:18:48:048	USER_DEBUG	[45] DEBUG Owner: John Doe
10:18:48:048	USER_DEBUG	[47] DEBUG Initial Balance: 5000.00
10:18:48:048	USER_DEBUG	[49] DEBUG Amount Deposited: 1500.00
10:18:48:049	USER_DEBUG	[51] DEBUG Total Balance: 6500.00
10:18:48:049	USER_DEBUG	[25] DEBUG Interest earned: 325.0000
10:18:48:049	USER_DEBUG	[27] DEBUG New Balance after interest: 6825.0000
10:18:48:049	USER_DEBUG	[5] DEBUG Total Balance after Interest: 6500.00
10:18:48:050	USER_DEBUG	[67] DEBUG Account Number: 1001
10:18:48:050	USER_DEBUG	[69] DEBUG Owner: John Doe
10:18:48:050	USER_DEBUG	[71] DEBUG Balance: 6500.00

Problem Statement

Create a class called **Shape** with a method called **getShapeArea()**.

Now, create a subclass called **RectangleShape** that overrides the **getShapeArea()** method to calculate the area of a rectangle.

```
public virtual class Shape {  
  
    public virtual Decimal getShapeArea() {  
  
        return 0;  
  
    }  
  
}
```

```

public class RectangleShape extends Shape {
    private Decimal length;
    private Decimal width;
    public RectangleShape(Decimal length, Decimal width) {
        this.length = length;
        this.width = width;
    }
    public override Decimal getShapeArea() {
        return length * width;
    }
}

```

```

RectangleShape rectangle = new RectangleShape(10.0, 5.0);
Decimal area = rectangle.getShapeArea();
System.debug('Area of the rectangle: ' + area);

```

Execution Log		
Timestamp	Event	Details
10:28:19:026	USER_DEBUG	[3] DEBUG Area of the rectangle: 50.00

Problem Statement

Create an interface called **DiscountCalculator** that defines a method for calculating discounts. Implement a class that uses this interface as **CustomerDiscount**.

Requirements:

- DiscountCalculator Interface: It includes a calculateDiscount method that takes an input parameter as originalAmount, which represents the original price of an item.

- - CustomerDiscount Class: The CustomerDiscount class implements the DiscountCalculator interface. It provides an implementation for the calculateDiscount method.
-

```
public interface DiscountCalculator {

    Decimal calculateDiscount(Decimal originalAmount);

}

public class CustomerDiscount implements DiscountCalculator {

    // discount rate 10%
    private Decimal discountRate;

    public CustomerDiscount(Decimal rate) {
        this.discountRate = rate;
    }

    public Decimal calculateDiscount(Decimal originalAmount) {

        Decimal discount = originalAmount * discountRate / 100;
        Decimal discountedPrice = originalAmount - discount;
        return discountedPrice;
    }

}
```

```
46 CustomerDiscount discountCalculator = new CustomerDiscount(10)
47 Decimal originalAmount = 100.00;
48 Decimal discountedPrice = discountCalculator.calculateDiscount
49
50 System.debug('Original Price: ' + originalAmount);
51 System.debug('Discounted Price: ' + discountedPrice);
52
```

Execution Log

Timestamp	Event	Details
10:25:10:025	USER_DEBUG	[5] DEBUG Original Price: 100.00
10:25:10:025	USER_DEBUG	[6] DEBUG Discounted Price: 90.00